

useStateMemory.ts

frontend/hooks | TypeScript

```

/**
 * useStateMemory.ts
 *
 * Agentic Failure Memory – the learning layer of BioSync.
 *
 * Records every AI intervention and its outcome, then makes that history
 * available to the AI prompt engine to avoid repeating failed strategies.
 *
 * Architecture:
 * - Interventions are stored as JSON "semantic commits" in localStorage
 * - Each commit records pre/post scores, outcome, and timestamp
 * - A session-scoped blacklist is derived from commits with FAILED outcome
 * - The AI system reads the blacklist before generating each response
 *
 * Why localStorage?
 * - Zero latency – no network round-trip
 * - Persists across browser sessions
 * - Privacy-preserving – never leaves the device
 * - Sufficient for session-length memory (hackathon scope)
 *
 * Production extension: replace localStorage with a write-ahead log (WAL)
 * synced to an encrypted backend for longitudinal personalization.
 */

import { useState, useCallback, useEffect } from 'react';

// ■■■ Types ■■■
export type InterventionType =
  | 'TASK_STRIPPING'
  | 'BREATHING_EXERCISE'
  | 'MICRO_BREAK'
  | 'ENVIRONMENTAL_RESET'
  | 'DEEP_WORK_PROMPT'
  | 'FOCUS_REFRAME'
  | 'ESCALATE_HUMAN';

export type InterventionOutcome = 'SUCCESS' | 'FAILED' | 'PENDING';

export interface InterventionCommit {
  id: string;
  sessionId: string;
  timestamp: string; // ISO 8601
  interventionType: InterventionType;
  interventionText: string; // The actual AI response shown to user
  preScore: number; // Score when intervention was triggered
  postScore: number | null; // Score measured 60s later (null if pending)
  delta: number | null; // postScore - preScore
  outcome: InterventionOutcome;
}

interface UseStateMemoryReturn {
  /** All commits for the current session */
  commits: InterventionCommit[];
  /** Intervention types that have FAILED this session – for AI blacklist */
  blacklist: InterventionType[];
  /** Record a new intervention (outcome will be PENDING until evaluated) */
  recordIntervention: (
    type: InterventionType,
    text: string,
    score: number
  ) => string; // Returns commit ID
  /** Update a commit's post-score and compute outcome */
  resolveIntervention: (id: string, postScore: number) => void;
}

```

[illegible]

```

const allCommits = loadCommits();
const otherSessions = allCommits.filter(c => c.sessionId !== sessionId);
persistCommits(otherSessions);
setCommits([]);
}, [sessionId]);

return { commits, blacklist, recordIntervention, resolveIntervention, clearSession };
}

// ■■■■ Helpers ■■■■
function getOrCreateSessionId(): string {
if (typeof window === 'undefined') return 'ssr';
const existing = sessionStorage.getItem(SESSION_ID_KEY);
if (existing) return existing;
const newId = `session_${Date.now()}_${Math.random().toString(36).slice(2, 8)}`;
sessionStorage.setItem(SESSION_ID_KEY, newId);
return newId;
}

function generateId(): string {
return `commit_${Date.now()}_${Math.random().toString(36).slice(2, 8)}`;
}

function loadCommits(): InterventionCommit[] {
if (typeof window === 'undefined') return [];
try {
const raw = localStorage.getItem(STORAGE_KEY);
return raw ? (JSON.parse(raw) as InterventionCommit[]) : [];
} catch {
return [];
}
}

function persistCommits(commits: InterventionCommit[]): void {
if (typeof window === 'undefined') return;
// Keep only the last 100 commits to avoid unbounded storage growth
const trimmed = commits.slice(-100);
localStorage.setItem(STORAGE_KEY, JSON.stringify(trimmed));
}

```